

Realistic Car Controller V3.5

First of all, thank you for purchasing and using Realistic Car Controller! This documentation will guide you to;

Contents

Realistic Car Controller V3.5	1
Contents	1
First to Do!.....	2
Script Execution Order	3
Overview	3
RCC_CarControllerV3.cs.....	4
RCC Settings	6
Configurable Ground Materials	7
RCC Scene Manager	8
Controller Types	8
Mobile Controller	9
About Mobile Usement On City Scene	10
Keyboard Shortcuts.....	10
RCC Camera.....	11
Record / Replay	12
Customization	13
How The Customization Panel Works	13
Credits	14
License.....	14

You can find more updated details on

<http://www.bonecrackergames.com/realistic-car-controller>

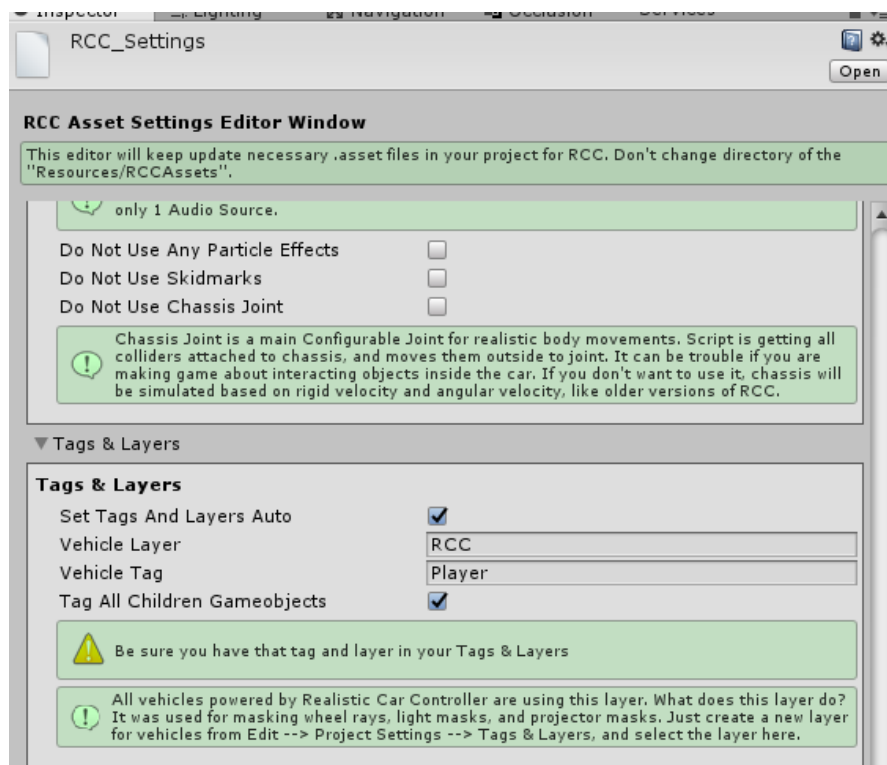
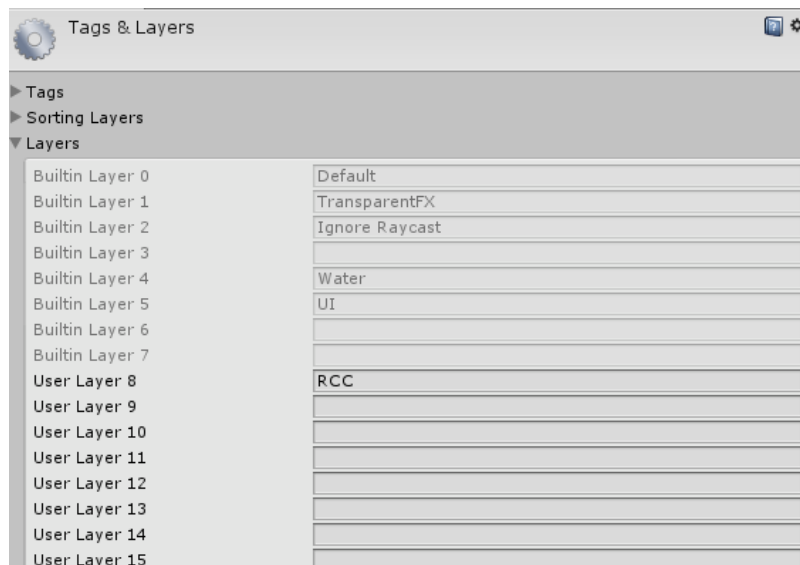
<https://www.youtube.com/playlist?list=PLRXTqAVrLDpoW58IKf8XA1AWD6kDkoKb1>

(You can zoom in with CTRL + ScrollUp for enlarge PDF pages)

First to Do!

Always backup your project before updating any asset or Unity Editor. Keep your own assets outside of the RealisticCarControllerV3 folder. Delete the entire folder, and import updated version.

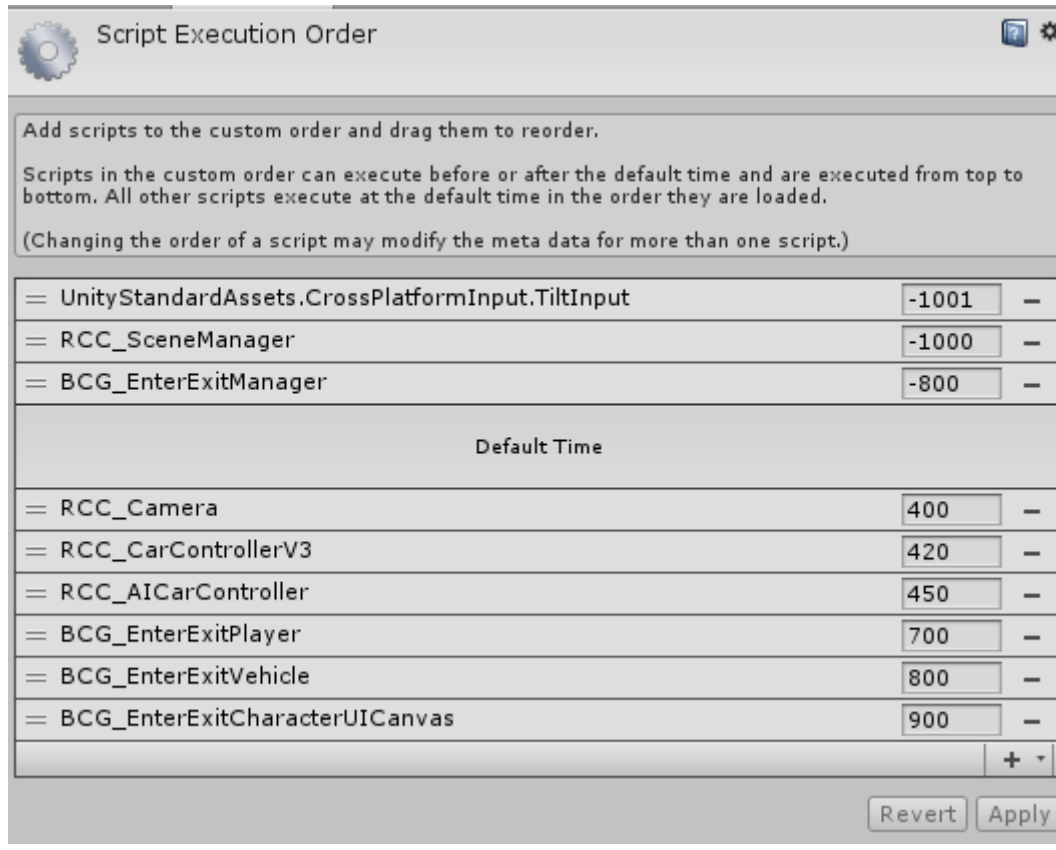
RCC is using **LayerMask** for avoid unwanted raycast hits, lens flare intensity and scale calculation, and ignore unnecessary projector layers. Create these layers and be sure they are selected in **RCC Settings**. Also you can import it from **Welcome Screen**, but it will overwrite your **Tags & Layers**.



(Tools → BoneCracker Games → Realistic Car Controller → RCC Settings)

Script Execution Order

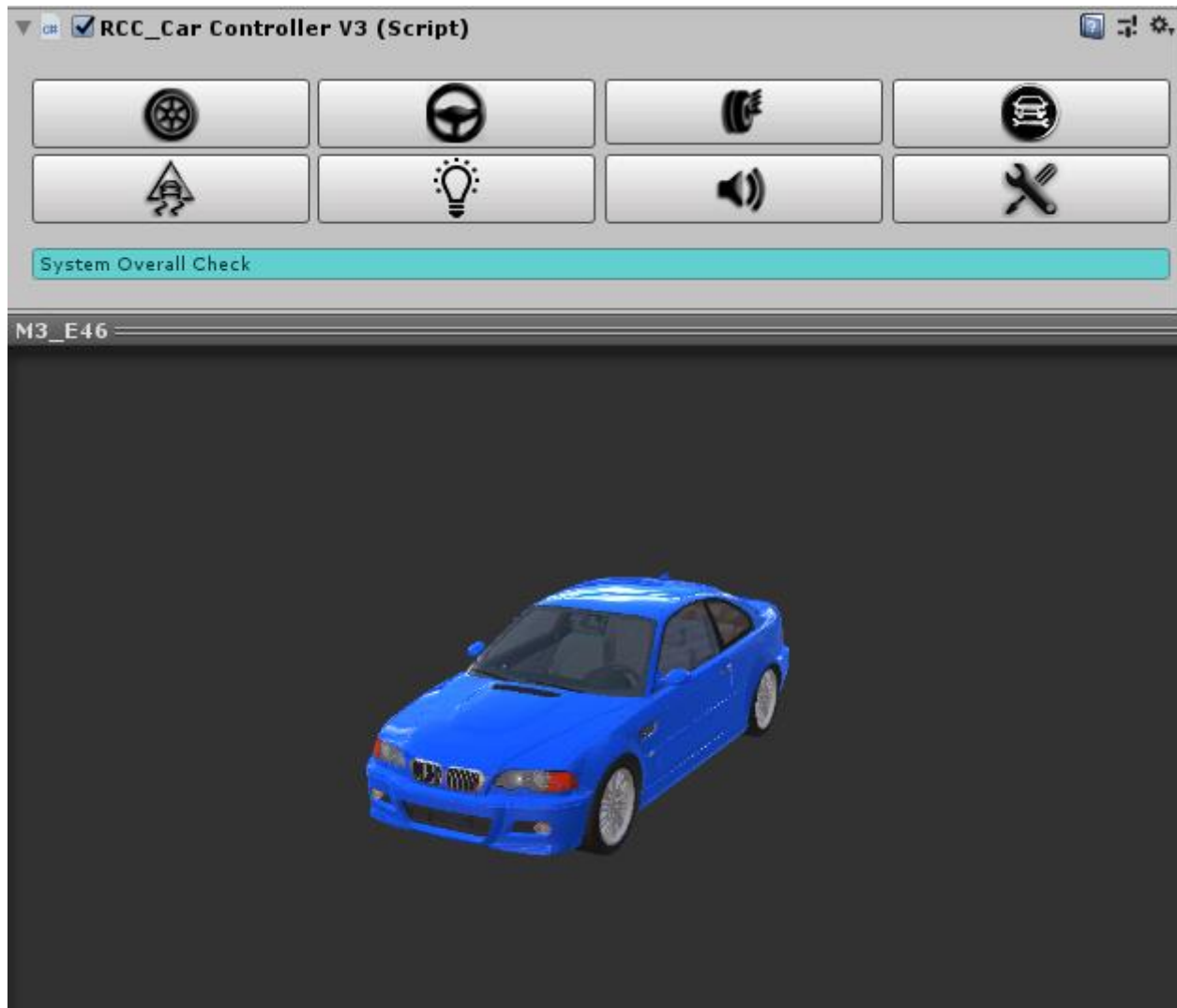
RCC is using [Script Execution Order](#) for avoid unexpected event conflicts. This should be imported successfully when RCC installed, and doesn't require any action. Just make sure you have this order. You can check it in [Edit → Project Settings → Script Execution Order](#).



Overview

Each vehicle has it's own [RCC_CarControllerV3.cs](#) script. Each vehicle is responsible for own [RCC_CarControllerV3.cs](#). All global shared settings are located in RCC Settings ([Tools → BCG → RCC → Edit Settings](#)). Lights, cameras, exhausts are addons and not required as an essential. Inputs are processed by [RCC_InputManager.cs](#) script. It will receive corresponding inputs from selected device. [RCC_SceneManager.cs](#) is managing active player vehicle, other vehicles, AI vehicles, record/replay, UI canvases, etc... All other main topics can be found below.

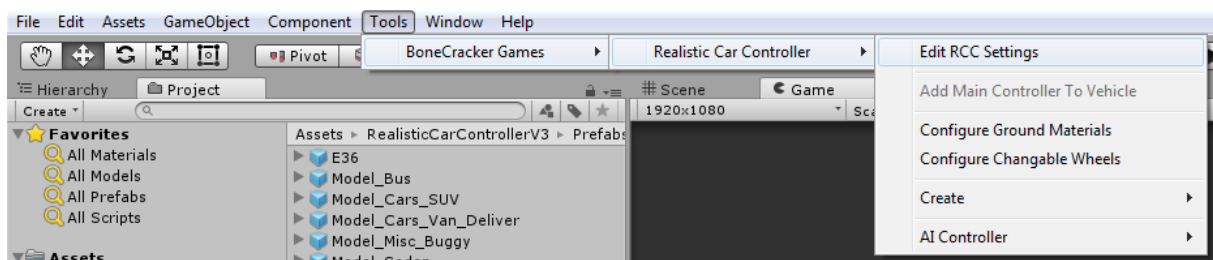
RCC_CarControllerV3.cs



8 Main Categories for easily and understandable creating / configuring vehicles.

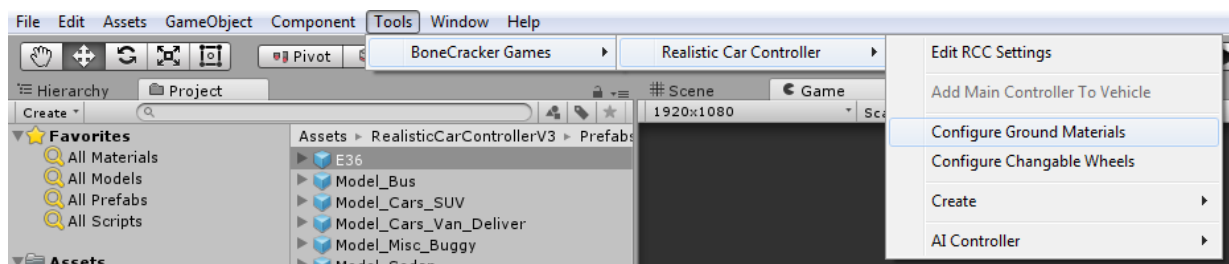
Wheels, Steering, Suspensions, Mechanic Configuration, Stability, Lights, Sounds, and Damage.

All vehicles are sharing global settings, sounds, configurations via **RCC Settings**.

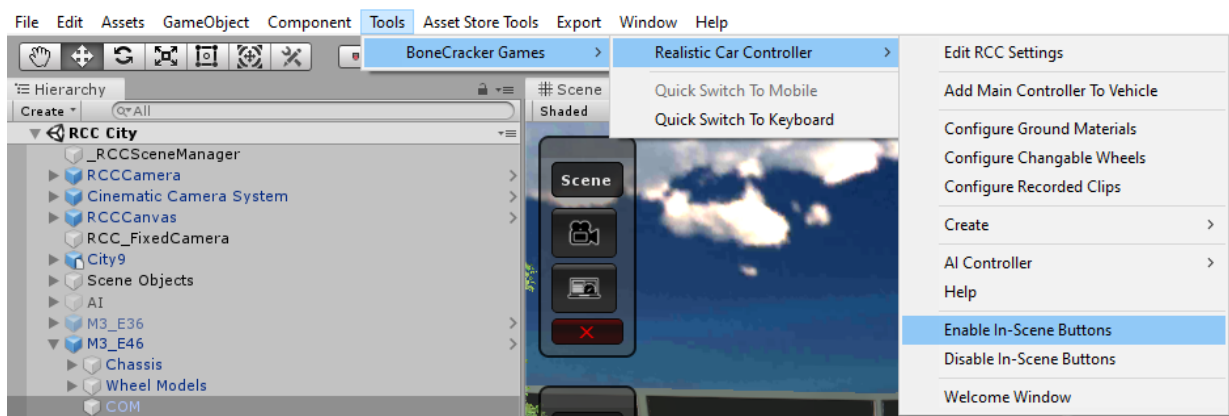


Creating new vehicles is explained in documentation named **“Realistic Car Controller V3.5 How To Create New Vehicles”**.

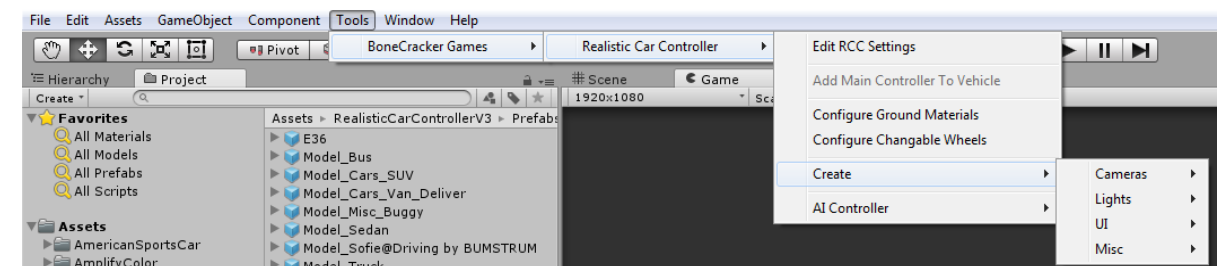
Changing ground materials physics, particles, sounds, etc in **Tools → BoneCracker Games → Realistic Car Controller → Configure Ground Materials**. (Detailed explanation in documentation named “Realistic Car Controller V3.5 RCC_GroundMaterials”)



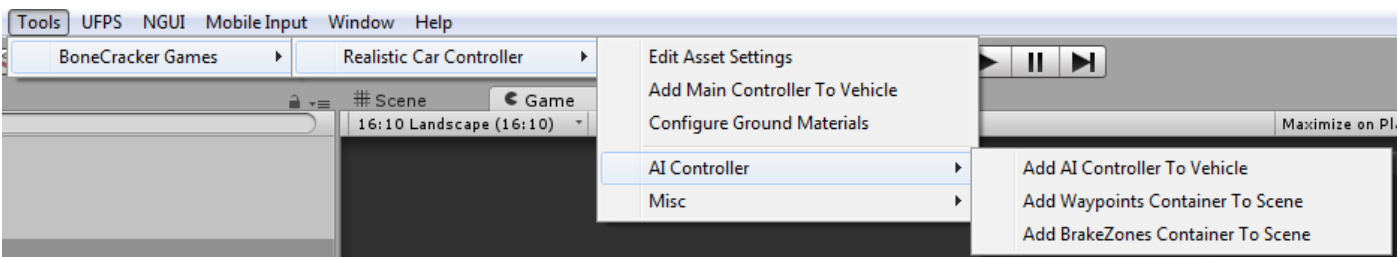
You may want to enable In-Scene buttons to create addons fastest way. **Tools → BoneCracker Games → Realistic Car Controller → Enable In-Scene Buttons**. (Detailed explanation in documentation named “Realistic Car Controller V3.5 How To Create New Vehicles”)



Creating lights, exhausts, mirrors, cameras, etc in **Tools → BoneCracker Games → Realistic Car Controller → Create**. (Detailed explanation in documentation named “Realistic Car Controller V3.5 How To Create New Vehicles”)

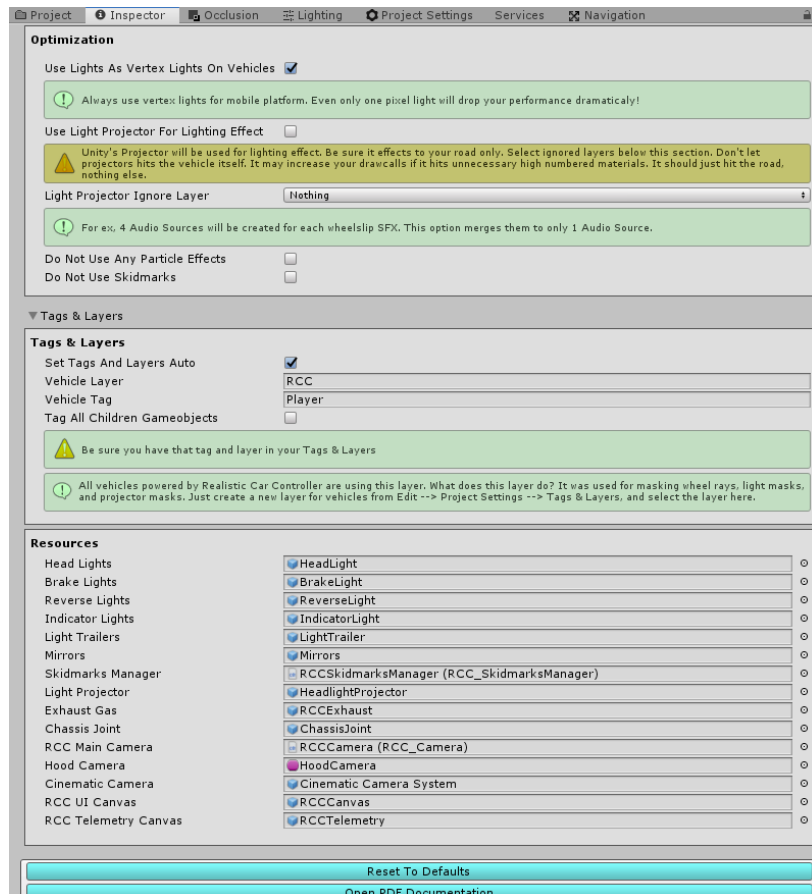
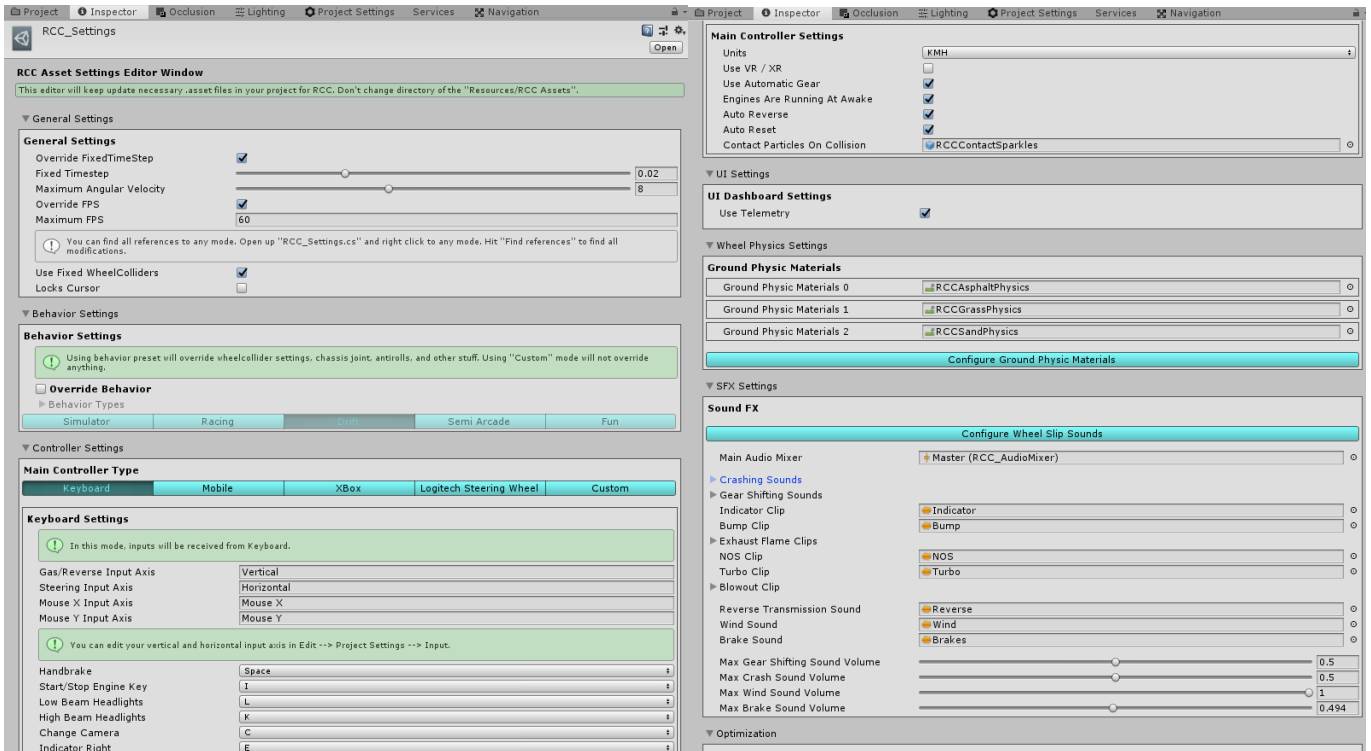


Making vehicles controlled by AI in **Tools → BoneCracker Games → Realistic Car Controller → AI Controller**. (Detailed explanation in documentation named “Realistic Car Controller V3.5 AI”)



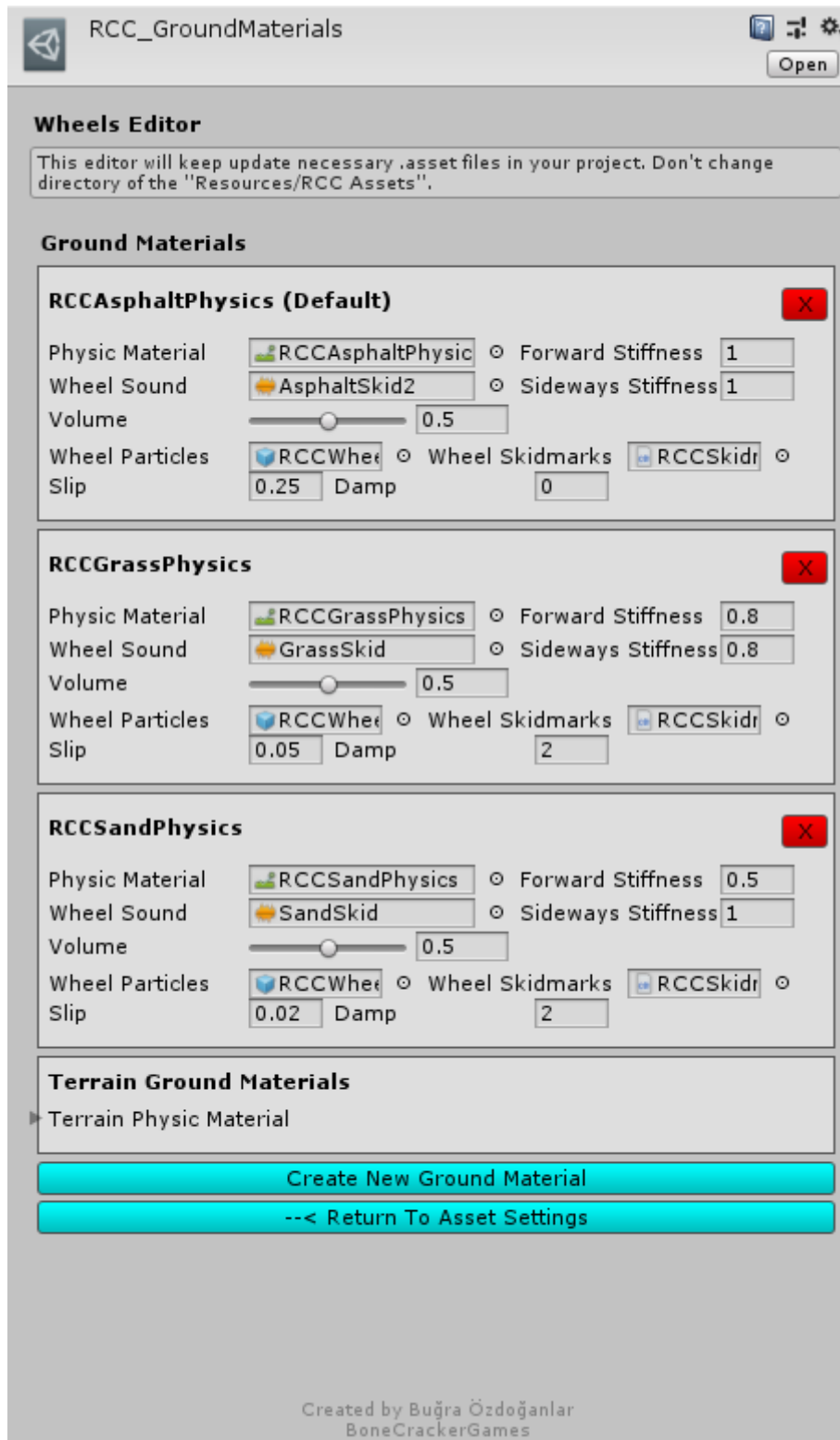
RCC Settings

Main RCC Settings. It's shared by all vehicles powered by RCC. [Tools → BoneCracker Games → Realistic Car Controller → RCC Settings](#). (Detailed explanation in documentation named “Realistic Car Controller V3.5 RCC_Settings”)



Configurable Ground Materials

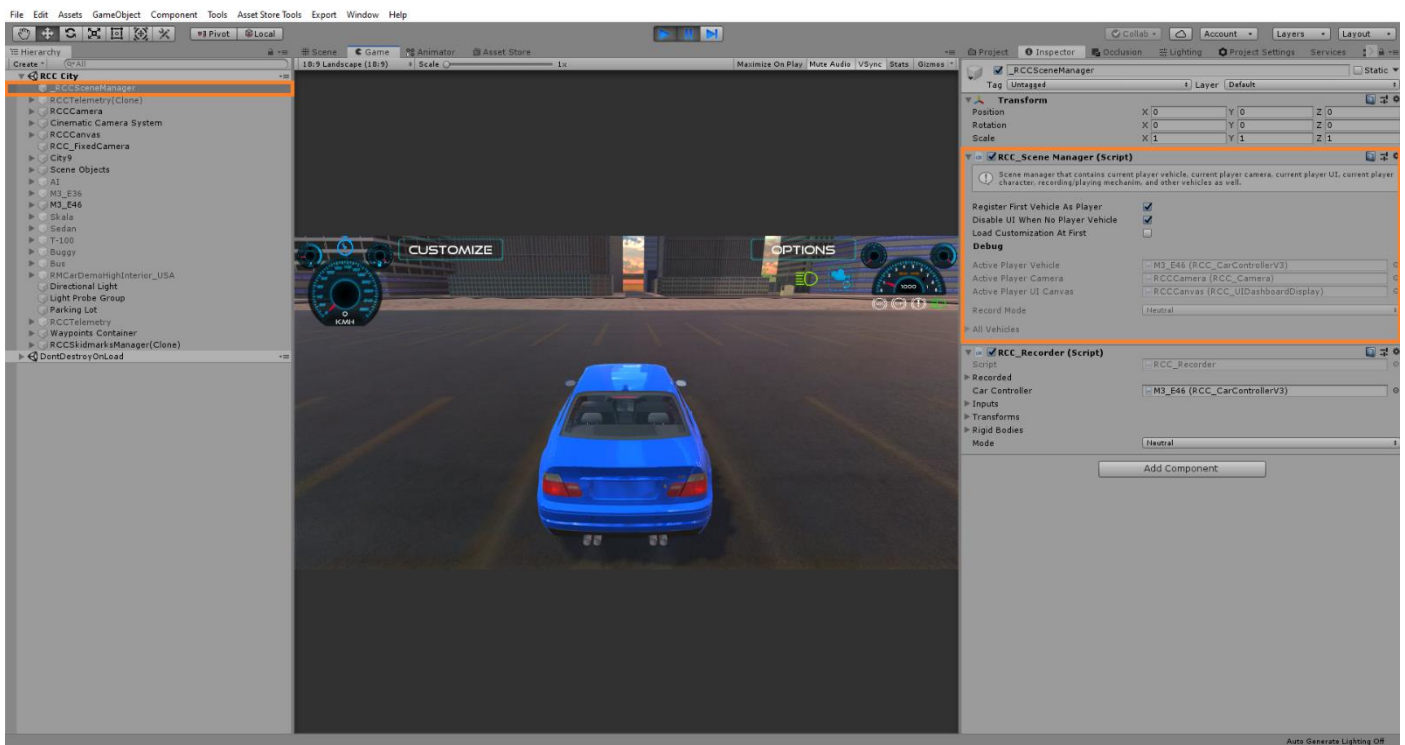
Changing or adding new ground materials physics, particles, damp, sounds, etc in [Tools → BoneCracker Games → Realistic Car Controller → Configure Ground Materials](#). (Detailed explanation in documentation named “Realistic Car Controller V3.5 RCC_GroundMaterials”)



If WheelCollider hits a collider with one of the physic material in list, changes will be applied to WheelCollider. You can check out demo scenes.

RCC Scene Manager

Every scene will have this manager automatically. **RCC Scene Manager** contains current player vehicle, current player camera, current player UI, current player character, recording / replay mechanism, and other vehicles as well. Instead of finding current car controller, or camera on scene, RCC Scene Manager will find it and manage it only. All other scripts depending on player vehicle will take reference of the RCC Scene Manager. For ex, finding player vehicle on scene is **RCC_SceneManager.Instance.activePlayerVehicle**. All other codes can be found at scripts documentation.



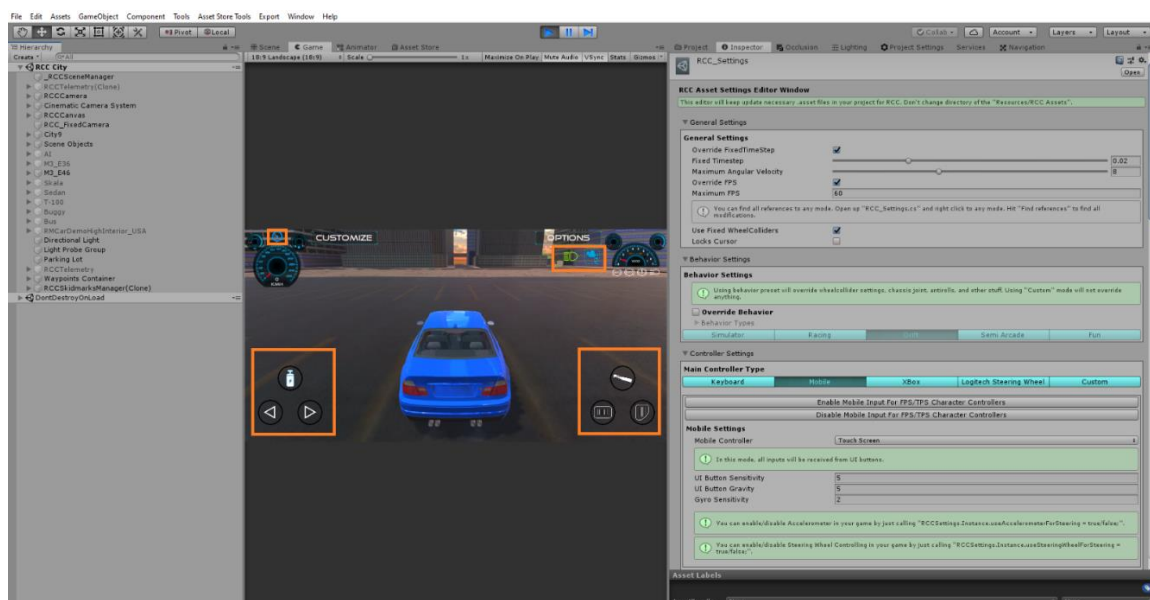
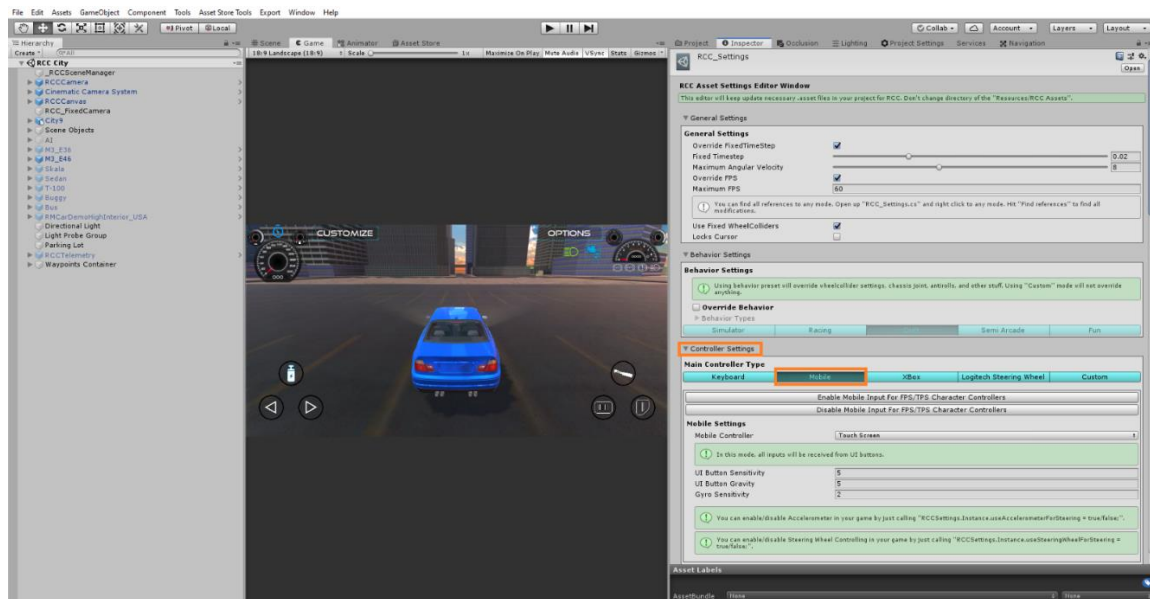
Controller Types

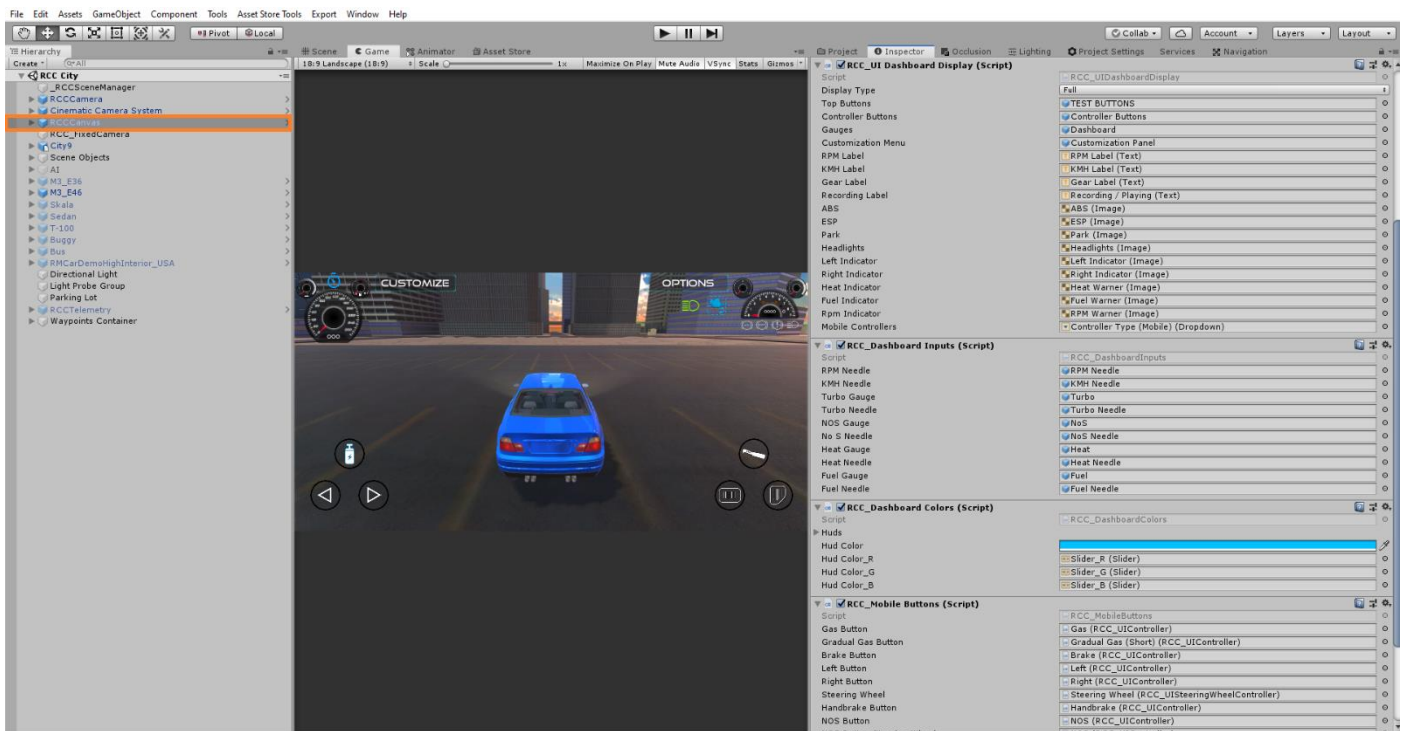
RCC supports all controller types with the new input system. Each controller can be changed directly from **RCC_InputActions** (Detailed explanation in documentation named “**Realistic Car Controller V3.5 New Input System**”)

Logitech Steering requires [Logitech Gaming SDK](#) installed in your project.

Mobile Controller

Mobile controller is using my own input system instead of the new input manager. Each UI controller button has “[RCC_UIController.cs](#)” script for inputs. These buttons feeds [RCC_InputManager](#) with normalized float values. You can adjust UI buttons sensitivity and gravity from [RCC Settings](#). Switching mobile controller to the new input manager is easy, however I don't recommend to do this. Because UI buttons will simulate gamepad buttons in that case.





About Mobile Usement On City Scene

City scene has lot of specular maps with alpha channels. Textures with alpha channels and bump maps are heavy for mobile devices. In Demo APK from my website is not using any texture with alpha channels. Also all standard shaders are replaced with mobile shaders in **RCC City Mobile** scene at the demo. If you build an APK without editing materials, you may get performance loss on low-end devices.

Keyboard Shortcuts

Shift + R = Add main controller to the vehicle

Shift + S = RCC Settings

Shift + E = Enable In-Scene editor buttons

Some packages may conflict with the shortcuts. For example, removing road key in EasyRoads (Shift + R) will conflict with this. To remove or change any shortcut, edit RCC_Editor.cs script;

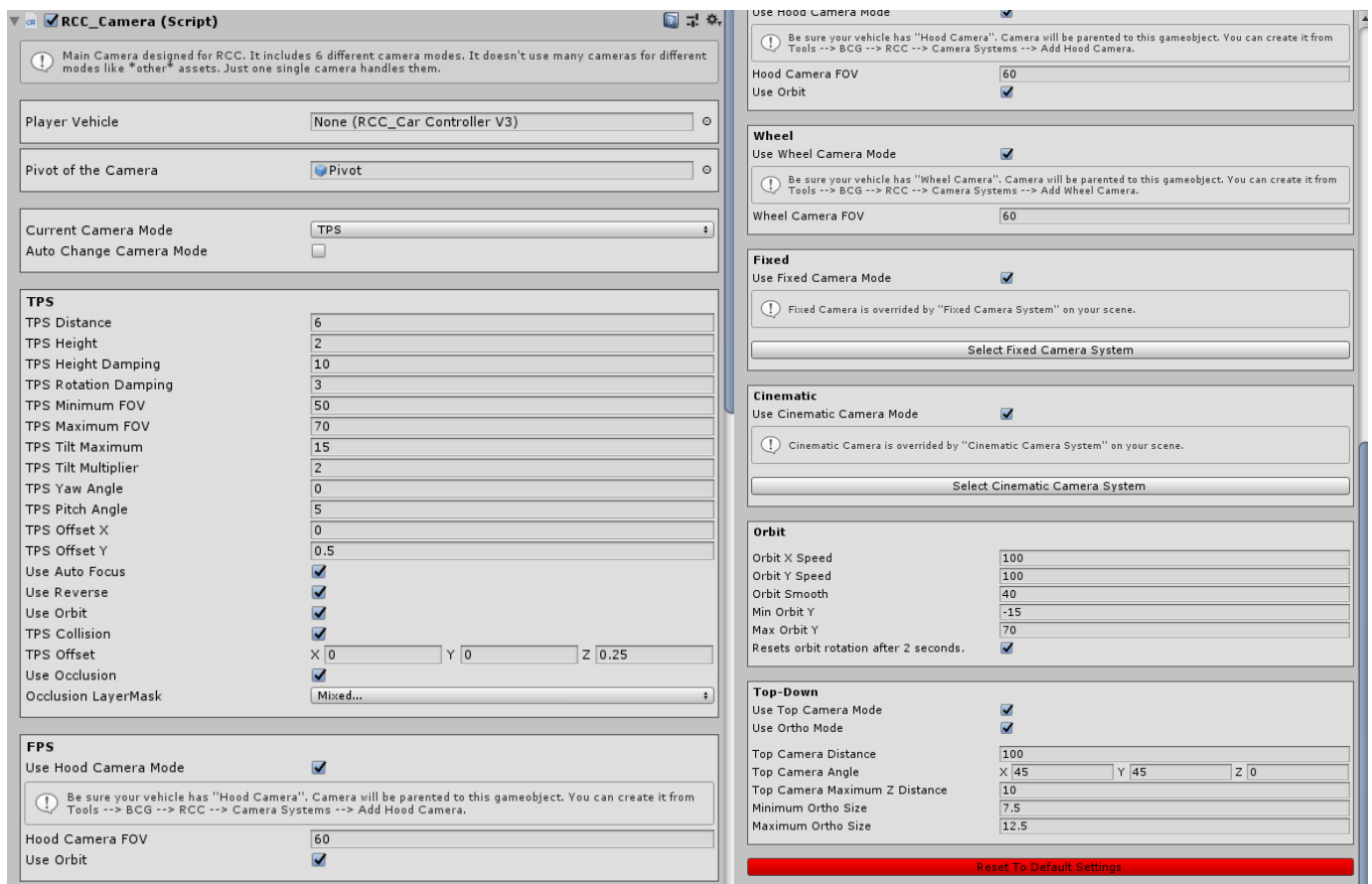
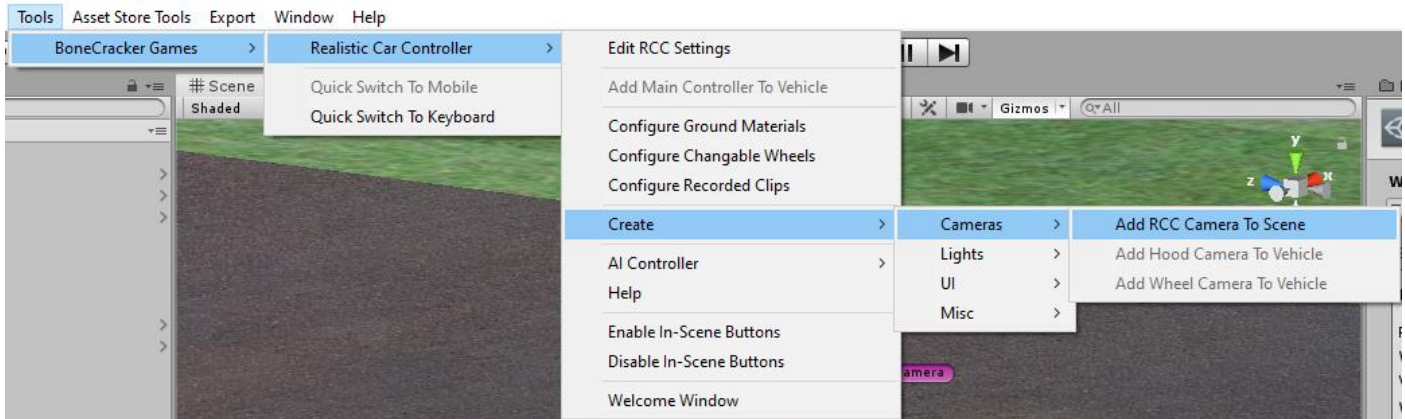
```
[MenuItem("Tools/BoneCracker Games/Realistic Car Controller/Add Main Controller  
To Vehicle #r", false, -85)]
```

#r is the key for shortcut. You can remove it or change it to any other button.

RCC Camera

Main camera system designed for using with RCC. Related with vehicle stats and includes six different camera modes with many customizable settings. It doesn't use different individual cameras on your scene. Simply it parents the camera to their positions, and that's all.

If your scene doesn't have RCC Camera, you can create it from [Tools → BoneCracker Games → Realistic Car Controller → Create → Cameras → Add RCC Camera To Scene](#).



Each camera mode can be customized here. TPS mode is required, and all other modes are optional. If you don't want to use hood, wheel, fixed, cinematic camera, top-down modes, you can just disable them here.

Record / Replay

Complete physics and input based record / replay system. Player vehicle and all active AI vehicles can record / replay. All you have to do is press “**R**” for start recording, and “**P**” for start replay. These buttons can be changed in [RCC_InputActions](#). And of course, there is a UI button for mobile.

[RCC_Recorder](#) can be found at attached to **_RCCSceneManager** on your scene. Script will be added at awake, or you can add it by manually. You can use RCC’s API for start record / replay at runtime. For ex;

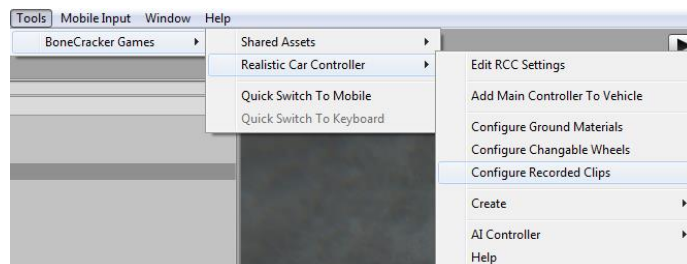
RCC. StartStopReplay ();

RCC. StartStopReplay (RCC_Recorder.Recorded recordedClip);

RCC. StartStopReplay (int index);

RCC. StartStopReplay (RCC_Recorder.Recorded recordedClip);

All records are stored in [RCC_Records](#). You can access it from **Tools → BCG → RCC → Configure Recorded Clips**.



Customization

You can customize your vehicles by just calling a single method. Please take a look at “[Realistic Car Controller V3 Scripts](#)” documentation. All methods in [RCC_Customization](#) are explained there.

How The Customization Panel Works

I wrote a example script called “[RCC_CustomizerExample.cs](#)” which uses static methods in [RCC_Customization.cs](#). Script is attached to **RCCCanvas**. UI Buttons in Customization Panel sends methods to this example script. And this example script uses static methods in [RCC_Customization.cs](#) for making changes. Let me explain it with simple examples;

We want to change front suspension distance of our vehicle. So, we have to call;

[RCC_Customization.SetFrontSuspensionsDistances](#) ([targetRCC](#), [targetValue](#));

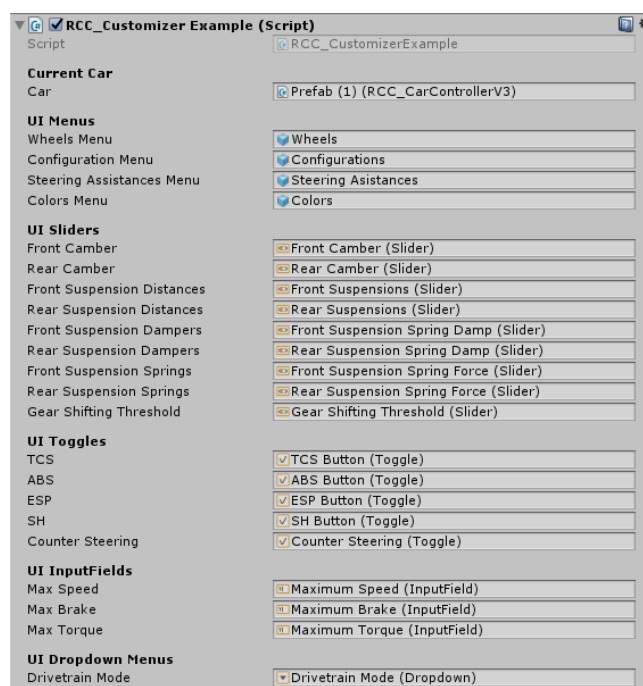
We want to repair our car. So, we have to call;

[RCC_Customization.RepairCar](#) ([targetRCC](#));

We want to change drivetrain of our car to AWD. So, we have to call it;

[RCC_Customization.SetDrivetrainMode](#) ([targetRCC](#), [RCC_CarControllerV3 WheelType.AWD](#));

And goes on... Simply take a look at all methods in [RCC_CustomizerExample.cs](#) script, you will see how did I modify the player vehicle by using [RCC_Customization.cs](#) script.



This example script handles all UI menus, buttons, sliders, toggles, inputfields, and dropdown menus of the customization panel. It just receives inputs from UI, and fires necessary actions.

Credits

Driver Sofie, her animations, and her car model made by 3DMAesen. You can access 3DMAesen asset store from this link;

<http://u3d.as/2vg>

All sounds in package are completely Royalt Free. You can use them on any personal or commercial projects. You can't redistribute / resell them.

License

You can use this package for unlimited games. Both personal and commercial use. **But you can't resell or redistribute any asset in the package on any**

store (not even any single asset in package). I got many reports from my customers about some fake developers are reselling my package on other stores. This is strictly forbidden. You can't resell or redistribute ANY asset from Unity's Asset Store, unless if developer gave you special license for making this. If anyone violates this, he will be banned, and his revenue from package sellings will be interrupted. You can read Unity EULA from this link;

http://unity3d.com/legal/as_terms

So, how you holding up there? You can ask me anything about my assets! If you want change **minor things** in the package, don't waste your time by editing scripts. Just tell me, I'll do my best with no cost. I don't take any projects right now, and I'm not available for hire. Please mail me if you used any of my assets in your game. I'd like to see it in action!

Ekrem Buğra Özdoğanlar

Bonecrackergames@gmail.com